the users while leaving the operational aspects to AWS.

Development teams can select the most suitable database for their specific application needs, rather than relying on multi-purpose databases. This shift allows better alignment between application requirements and database capabilities.

Open source databases supported by Amazon RDS

Amazon RDS is a managed service provided by AWS for launching and managing relational databases designed for online transaction processing (OLTP) and serves as a drop-in replacement for on-premises database instances. Key features include automated backups, patching, scaling and redundancy.

Amazon RDS supports various open source database engines such as MySQL, PostgreSQL, and MariaDB. The service provides security, easy scaling for storage and compute, multi-AZ deployments for high availability, automatic failover, and read replicas for read-heavy workloads.

A DB instance in Amazon RDS represents a database environment in the cloud with specified compute and storage resources. Database instances are accessed via endpoints, which can be obtained through various methods. Customers can have up to 40 RDS DB instances by default for all three open source databases specified. Maintenance windows can be configured for modifications, and customers can define their preferred window or AWS can schedule a 30-minute window.

Open source NoSQL databases provided by Amazon EMR

Amazon EMR (Elastic MapReduce), a managed service for processing and analysing large amounts of data, supports more than thirty plus open source Big Data frameworks including Apache Hadoop, HBase and Apache Spark. It simplifies the deployment and management of these frameworks, allowing users to focus on data processing tasks without worrying about infrastructure. EMR seamlessly integrates with Amazon S3, enabling easy data ingestion and storage. Users can read data from S3 directly into EMR, process it, and write the results back to S3, which makes EMR and S3 a powerful combination for building data lakes. Running clusters can be reconfigured.

Amazon EMR platform service allows users to provision and manage clusters of virtual servers (EC2 instances) on which the Big Data processing frameworks run. Users can define the cluster size, instance types, and other configurations to meet their specific requirements.

Amazon EMR Serverless is an extension of Amazon EMR that offers a serverless Big Data processing option. It allows Apache Spark, Apache Hive, and HBase to run on-demand without the need to provision or manage clusters manually. It automatically scales resources based on the workload and charges only for the actual usage.

EMR on EKS (Elastic Kubernetes Service) is a relatively new offering that helps to run Apache Spark on Amazon EKS, a managed Kubernetes service. With EMR on EKS, scalability and flexibility of Kubernetes to manage and scale Spark workloads is leveraged, integrating with other AWS services.

Apache HBase on Amazon EMR

Two storage modes (S3 and HDFS) are available for EMR managed HBase. We will discuss only the S3 storage mode.

By leveraging Apache HBase on Amazon EMR, you can benefit from simplified administration, flexible deployment, unlimited scalability, seamless integration with other AWS services, and reliable built-in backup functionality.

Apache HBase follows a 'bytes-in/bytes-out' interface, which means that any data that can be converted into an array of bytes can be stored as a value in HBase. This allows for flexibility in storing various data types, such as strings, numbers, complex objects, or even images.

Apache HBase reads and writes are highly consistent. In this service, the basic unit is a column. One or more columns form a row. Each row is addressed uniquely by a primary key referred to as a row key. A row in Apache HBase can have millions of columns. Each column can have multiple versions with each distinct value contained in a separate cell.

A column family is a container for grouping sets of related data together within one table, as shown in Figure 6.

Column families allow you to fetch only those columns that are required by a query. All members of a column family are physically stored together on a disk. This means that optimisation features, such as performance tunings, compression encodings, and so on, can be scoped at the column family level.

A row key can consist of multiple parts concatenated to provide an

Concept	Description
Row	Represents an atomic byte array or key/value container. Each row is uniquely identified by a row key.
Column	Refers to a key within the key/value container inside a row.
Column Family	Logically divides columns into related subsets of data stored together ondisk. Column families are defined at the table scherna level.
Tirnestamp	An explicit or implicit value associated with a cell, typically a long integer in milliseconds. Enables time-based versioning of cell values.
Value	Represents the actual data stored in a cell. A cell can contain multiple versions of a value, ordered by timestamp (most recent first).

Figure 5: Apache HBase